



# Recursion

**Mahir Labib Dihan**  
**Lecturer, CSE, BUET**



# Table of Contents

1. What is Recursion?
2. How Recursion Works: The Call Stack
3. Base Case and Recursive Case
4. Example: Factorial
5. Example: Fibonacci
6. Practice Problems
7. Summary

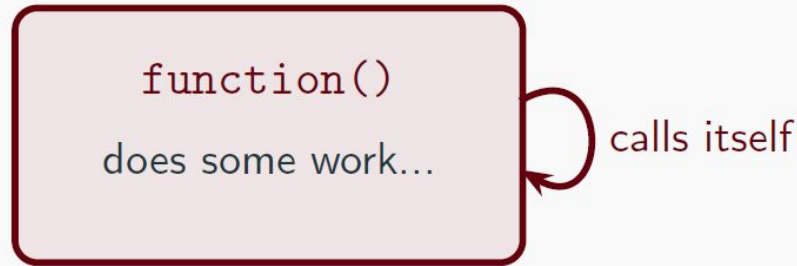
# What is Recursion?



# What is Recursion?

## Definition

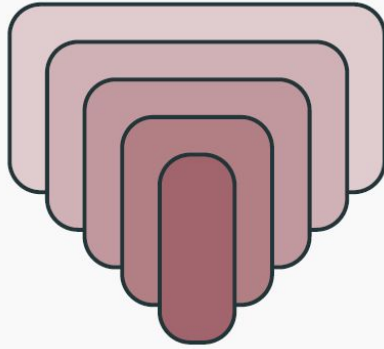
**Recursion** is when a function calls itself to solve a smaller version of the same problem.



A recursive function

**Key idea:** Break a big problem into smaller, similar sub-problems.

# Real-World Analogy: Russian Nesting Dolls



To open the outer doll,  
you must first open  
the doll inside it...

**Until you reach the  
smallest doll!**

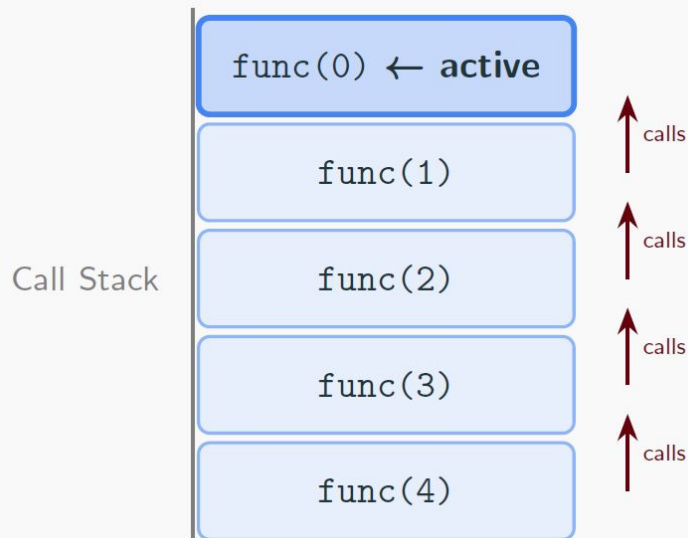
- Each doll contains a smaller version of itself
- The smallest doll is the **base case** — it doesn't contain another doll

# How Recursion Works: The Call Stack



# The Call Stack

When a function calls itself, each call is placed on a stack:



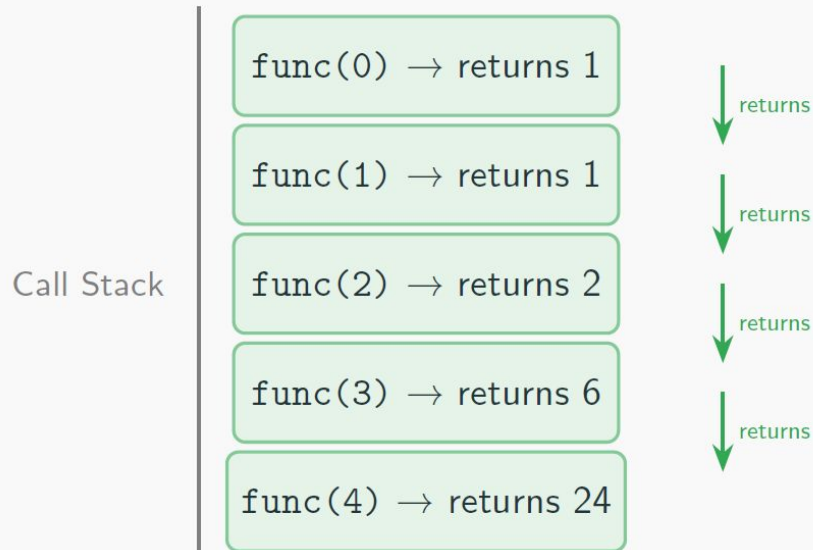
## Winding Phase:

Each call waits for the one above it to finish.

The topmost call is currently **executing**.

# Unwinding the Stack

When the base case is reached, the calls **return** one by one:



## Unwinding Phase:

Each call **returns** its result to the caller.

Values flow **back down** the stack.

# Base Case and Recursive Case



# Every Recursive Function Needs Two Parts

## 1. Base Case

The **stopping condition**.

Returns a value directly without recursion.

## 2. Recursive Case

Calls the function again with a **smaller/simpler** input.

```
int func(int n) {  
    // Base case  
    if (n == 0) {  
        return 1;  
    }  
    // Recursive case  
    else {  
        return n * func (n - 1);  
    }  
}
```

Without a base case, recursion would continue forever! (Stack Overflow)

# Example: Factorial



# Factorial: Mathematical Definition

**Factorial** of  $n$  (written  $n!$ ) is the product of all positive integers up to  $n$ .

$$n! = n \times (n - 1) \times (n - 2) \times \cdots \times 2 \times 1$$

## Examples:

- $4! = 4 \times 3 \times 2 \times 1 = 24$
- $3! = 3 \times 2 \times 1 = 6$
- $1! = 1$
- $0! = 1$  (by definition)

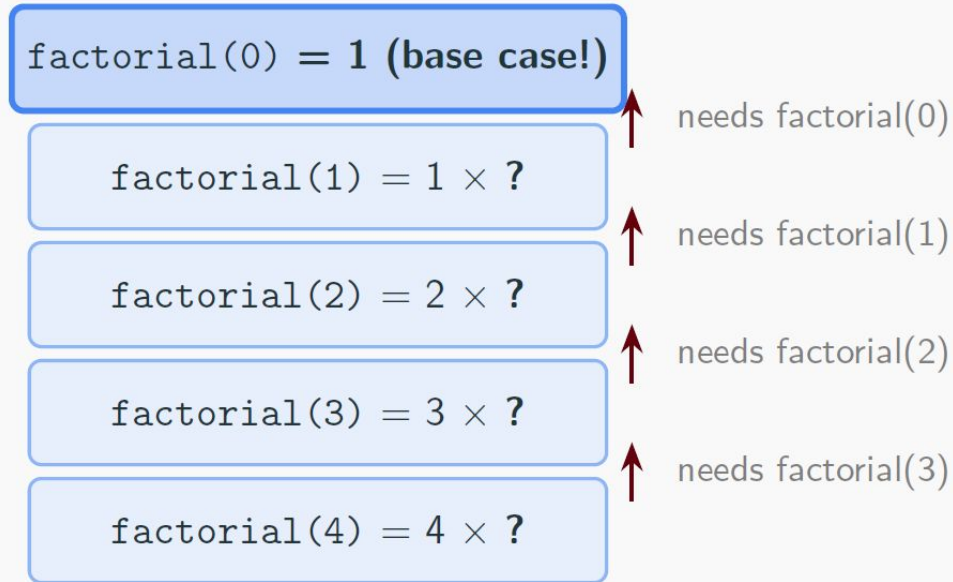
**Recursive definition:**  $n! = n \times (n - 1)!$  with base case  $0! = 1$

# Factorial: Recursive Code

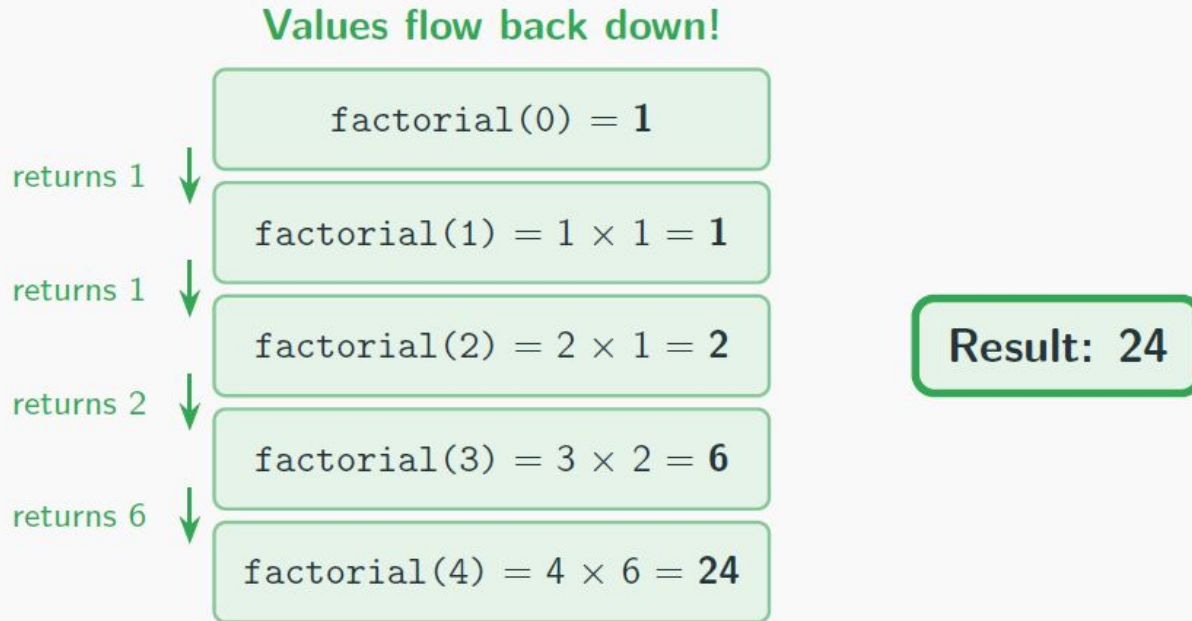
```
int factorial(int n) {  
    // Base case  
    if (n == 0) {  
        return 1;  
    }  
    // Recursive case  
    else {  
        return n * factorial (n - 1);  
    }  
}  
  
int main () {  
    int result = factorial(4);  
    printf("%d\n", result); // Output : 24  
    return 0;  
}
```

# Tracing factorial(4): Winding Phase

Each call waits for the answer...



# Tracing factorial(4): Unwinding Phase

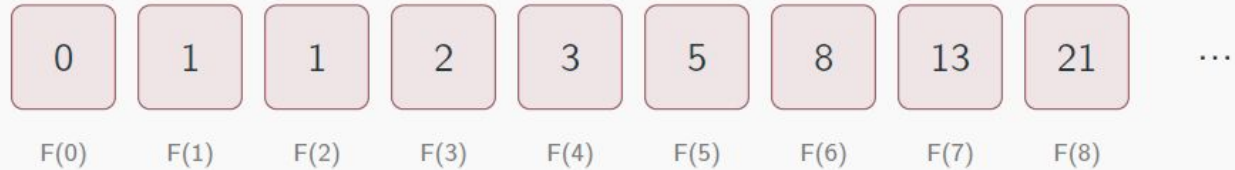


# Example: Fibonacci



# Fibonacci Sequence

The Fibonacci sequence: each number is the sum of the two before it.



## Recursive Definition:

- $F(0) = 0$  (base case)
- $F(1) = 1$  (base case)
- $F(n) = F(n - 1) + F(n - 2)$  for  $n \geq 2$  (recursive case)

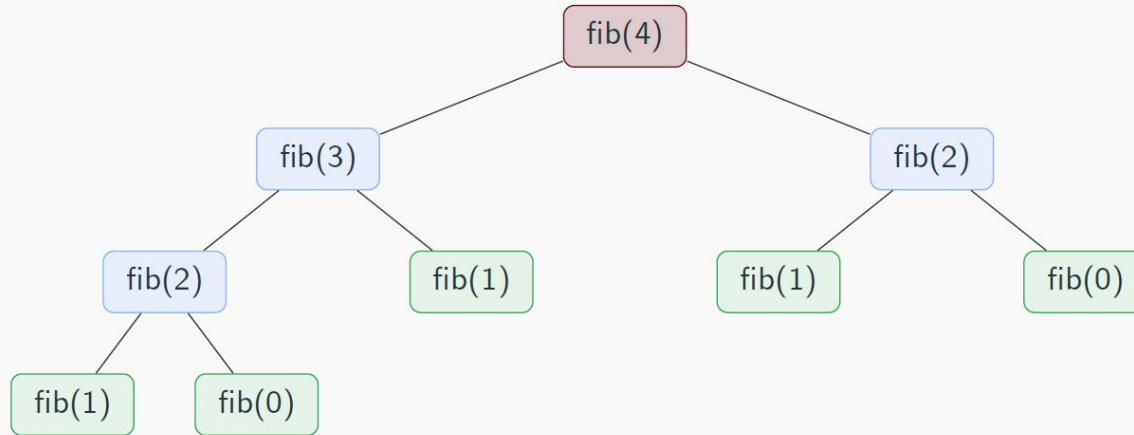
# Fibonacci: Recursive Code

```
int fib(int n) {  
    // Base cases  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
  
    // Recursive case : sum of two previous  
    return fib(n - 1) + fib(n - 2);  
}  
  
int main() {  
    printf("%d\n", fib(5)); // Output : 5  
    return 0;  
}
```

**Note:** This makes TWO recursive calls per invocation!

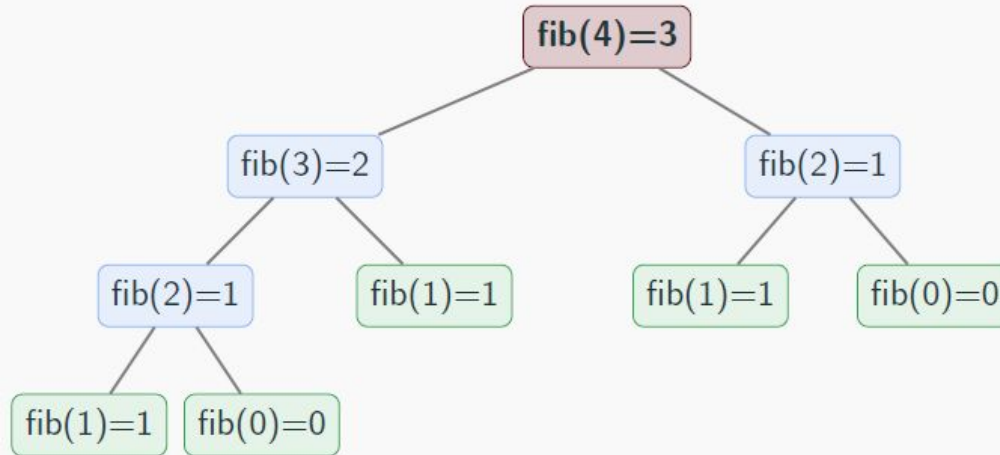
# Fibonacci Creates a Tree of Calls

Unlike factorial (linear), Fibonacci creates a **tree** structure:



**Green nodes** = base cases:  $\text{fib}(0)=0$ ,  $\text{fib}(1)=1$

# Tracing fib(4): Computing the Result



## Bottom-up:

$\text{fib}(0)=0, \text{fib}(1)=1$

$\text{fib}(2) = 1+0 = 1$

$\text{fib}(3) = 1+1 = 2$

$\text{fib}(4) = 2+1 = 3$

# Practice Problems



# Practice Problem 1: Sum of First N Numbers

**Problem:** Write a recursive function to compute  $1 + 2 + 3 + \dots + n$

**Example:** `sum(4)` should return  $1 + 2 + 3 + 4 = 10$

**Hints:**

- Base case: What is `sum(1)`?
- Recursive case:  $\text{sum}(n) = n + \text{sum}(n-1)$

**Solution:**

```
int sum(int n) {  
    if (n == 1) return 1; // Base case  
    return n + sum(n - 1); // Recursive case  
}
```

# Practice Problem 2: Power Function

**Problem:** Write a recursive function to compute  $x^n$  (x raised to power n)

**Example:** `power(2, 4)` should return  $2^4 = 16$

**Hints:**

- Base case: What is  $x^0$ ?
- Recursive case:  $x^n = x \times x^{n-1}$

**Solution:**

```
int power (int x, int n) {  
    if (n == 0) return 1; // Base case :  $x^0 = 1$   
    return x * power(x, n - 1); // Recursive case  
}
```

# Practice Problem 3: Count Digits

**Problem:** Write a recursive function to count digits in a positive integer

**Example:** `countDigits(1234)` should return 4

**Hints:**

- **Base case:** Single digit numbers have 1 digit
- **Recursive case:** Remove last digit using  $n/10$

**Solution:**

```
int countDigits(int n) {  
    if (n < 10) return 1; // Base case : single digit  
    return 1 + countDigits(n / 10); // Remove last digit  
}
```

# Summary



# Summary

## Key Concepts:

- Recursion = function calling itself
- Each call goes on the **call stack**
- **Winding:** calls build up
- **Unwinding:** returns flow back

## Structure:

- **Base case:** stops recursion
- **Recursive case:** makes progress toward base case

## Common Mistakes:

- Forgetting base case (infinite loop!)
- Not making progress toward base case
- Too deep recursion (stack overflow)

## When to use:

- Problem can be broken into similar sub-problems
- Tree/graph traversals
- Divide and conquer algorithms

# Acknowledgement

- Mohammad Sadat Hossain, Lecturer, CSE, BUET